

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Pattern Recognition and Text Processing

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Pattern Recognition and Text Processing
Weightage:	40% of CIA		
CO5 Statement:	Analyse regular expression patterns. (BL-3)		
Student Name:	NITHOSH P	Reg. No.:	192472269
Section / Batch:	BE CSE(AI) 3 rd yr	Date:	15/7/2026

Code of Conduct

I NITHOSH P (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Show all intermediate steps, calculations, state transitions, and reasoning wherever applicable.
- Use only the Python re (Regular Expressions) module to solve the given problems.
- Clearly mention the Regular Expression (Regex) pattern used before writing the corresponding Python program.
- Display the required output for each question, including extracted values, validation results, counts, and positions wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Generation	Pattern Matching Information Extraction	1
Search for a String	Keyword Search and Text Analysis	2
Split the documentation into sentences and words	Text Tokenization and Sentence Segmentation	3
Email and Strong Password Validation	Data Validation and Format Verification	4
Compile Once, Validate Many	Pattern Reusability and Performance Optimization	5

Section 1: Regular Expression Pattern Generation

Student Details:

Name: John Doe

Email: john.doe123@gmail.com

Mobile: +91-9876543210

Password: P@ssw0rd123

Date of Birth: 15/08/2004

Register Number: 23AIML1056

Department: Artificial Intelligence and Machine Learning

For the given student details formulate Regex patterns for the following:

Email ID

Mobile Number

Strong Password

Date of Birth (DD/MM/YYYY)

Register Number

ANSWER:

```
import re
```

```
# Sample student details text
```

```
student_data = (
```

```
    "Name: John Doe ,Email: john.doe123@gmail.com ,Mobile: +91-9876543210 ,"
```

```
    "Password: P@ssw0rd123 ,Date of Birth: 15/08/2004 ,Register Number: 23AIML1056 ,"
```

```
    "Department: Artificial Intelligence and Machine Learning"
```

```
)
```

```
# Define regex patterns
```

```
patterns = {
```

```
    "Email ID": r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}',
```

```
    "Mobile Number": r'\+\d{2}-\d{10}',
```

```
    "Strong Password": r'(?<=Password:\s)\S+',
```

```
    "Date of Birth": r'\b\d{2}/\d{2}/\d{4}\b',
```

```
    "Register Number": r'\b\d{2}[A-Z]{4}\d{4}\b'
```

```
}
```

```
print("--- Section 1: Extracted Student Details ---")
```

```
for key, pattern in patterns.items():
```

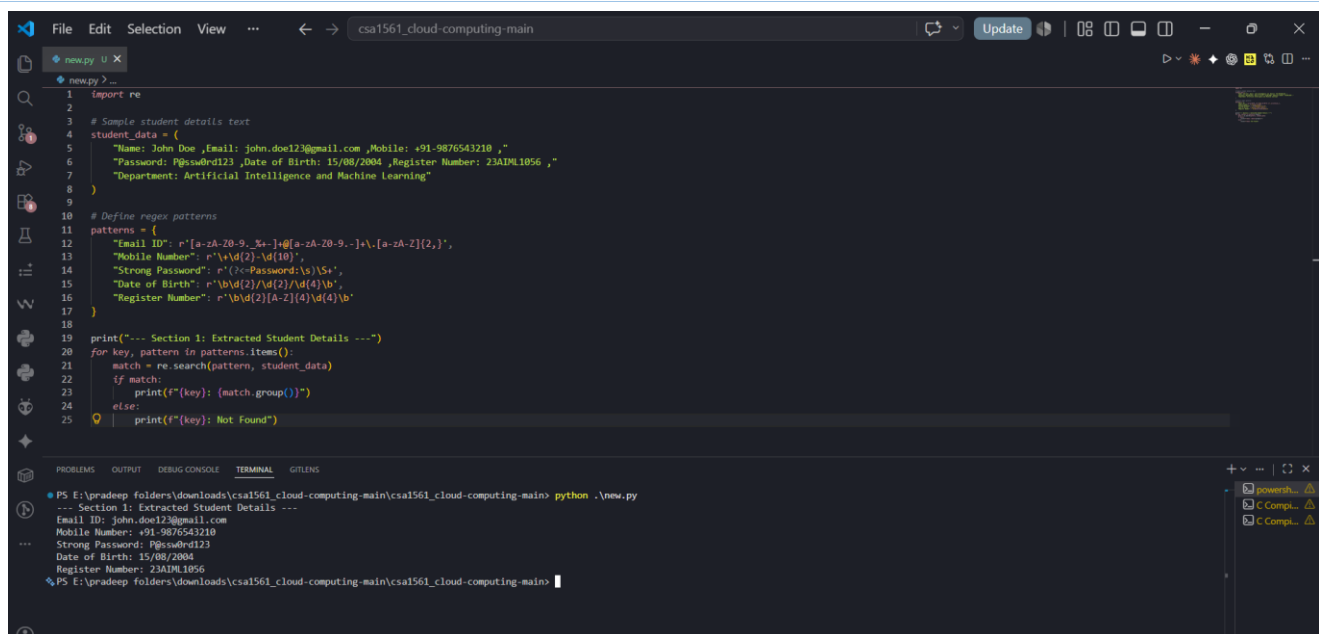
```
    match = re.search(pattern, student_data)
```

```
    if match:
```

```
        print(f"{key}: {match.group()}")
```

```
    else:
```

```
        print(f"{key}: Not Found")
```



```
1 import re
2
3 # Sample student details text
4 student_data = {
5     "Name: John Doe ,Email: john.doe123@gmail.com ,Mobile: +91-9876543210 ,"
6     "Password: Pqksw0rd123 ,Date of Birth: 15/08/2004 ,Register Number: 23ADM1056 ,"
7     "Department: Artificial Intelligence and Machine Learning"
8 }
9
10 # Define regex patterns
11 patterns = {
12     "Email ID": r"[a-zA-Z0-9_%.]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}",
13     "Mobile Number": r"\d{2}-\d{10}",
14     "Strong Password": r"(?=.*Password:\S+)\S+",
15     "Date of Birth": r"\b\d{2}/\d{2}/\d{4}\b",
16     "Register Number": r"\b\d{2}[A-Z]{4}\d{4}\b"
17 }
18
19 print("--- Section 1: Extracted Student Details ---")
20 for key, pattern in patterns.items():
21     match = re.search(pattern, student_data)
22     if match:
23         print(f"{key}: {match.group()}")
24     else:
25         print(f"{key}: Not Found")
```

```
PS E:\pradeep\folders\downloads\csa1561_cloud-computing-main\csa1561_cloud-computing-main> python .\new.py
--- Section 1: Extracted Student Details ---
Email ID: john.doe123@gmail.com
Mobile Number: +91-9876543210
Strong Password: Pqksw0rd123
Date of Birth: 15/08/2004
Register Number: 23ADM1056
PS E:\pradeep\folders\downloads\csa1561_cloud-computing-main\csa1561_cloud-computing-main>
```

Section 2: Search for a String

Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand.

Write a Python program using the re module to perform the following operations for the word "AI":

1. Find the first occurrence of the word "AI" using re.search().
2. Display the starting position of the first occurrence.
3. Display the ending position of the first occurrence.
4. Display the total number of times the word "AI" appears in the passage.
5. Print an appropriate message if the word is not found.

ANSWER:

```
import re
```

```
passage = (
    "Artificial Intelligence (AI) is transforming industries across the world. "
    "AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, "
    "and in education to provide personalized learning experiences. Many companies "
    "invest heavily in AI research because AI improves efficiency and enables intelligent "
    "decision-making. As AI continues to evolve, professionals with AI skills are in high demand."
)
```

```
pattern = r'\bAI\b'
```

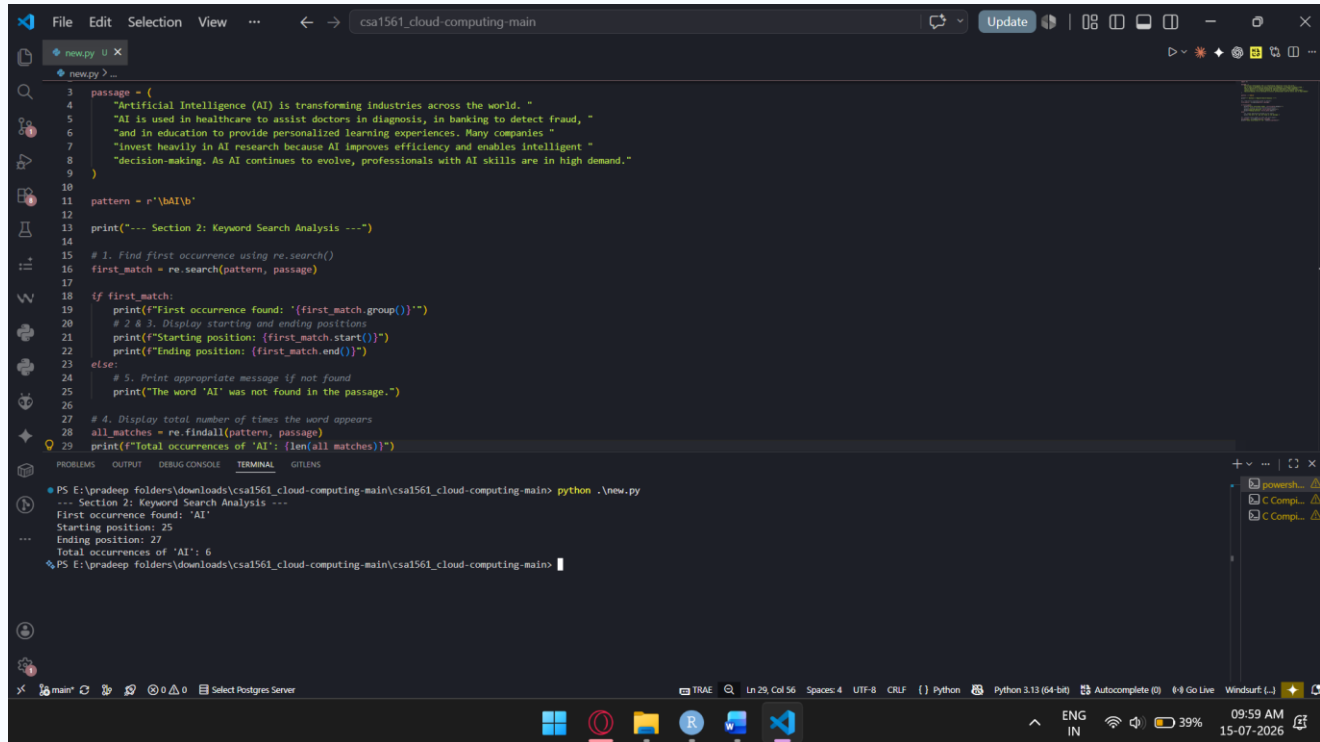
```
print("--- Section 2: Keyword Search Analysis ---")
```

```
# 1. Find first occurrence using re.search()
first_match = re.search(pattern, passage)
```

```
if first_match:
    print(f"First occurrence found: {first_match.group()}")
    # 2 & 3. Display starting and ending positions
    print(f"Starting position: {first_match.start()}")
    print(f"Ending position: {first_match.end()}")
else:
    # 5. Print appropriate message if not found
    print("The word 'AI' was not found in the passage.")
```

```
# 4. Display total number of times the word appears
all_matches = re.findall(pattern, passage)
```

```
print(f"Total occurrences of 'AI': {len(all_matches)}")
```



```
3 passage = {
4     "Artificial Intelligence (AI) is transforming industries across the world. "
5     "AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, "
6     "and in education to provide personalized learning experiences. Many companies "
7     "invest heavily in AI research because AI improves efficiency and enables intelligent "
8     "decision-making. As AI continues to evolve, professionals with AI skills are in high demand."
9 }
10
11 pattern = r'\bAI\b'
12
13 print("--- Section 2: Keyword Search Analysis ---")
14
15 # 1. Find first occurrence using re.search()
16 first_match = re.search(pattern, passage)
17
18 if first_match:
19     print(f"First occurrence found: '{first_match.group()}'")
20     # 2 & 3. Display starting and ending positions
21     print(f"Starting position: {first_match.start()}")
22     print(f"Ending position: {first_match.end()}")
23 else:
24     # 5. Print appropriate message if not found
25     print("The word 'AI' was not found in the passage.")
26
27 # 4. Display total number of times the word appears
28 all_matches = re.findall(pattern, passage)
29 print(f"Total occurrences of 'AI': {len(all_matches)}")
```

PS E:\pradeep\folders\downloads\csa1561_cloud-computing-main\csa1561_cloud-computing-main> python .\new.py

```
--- Section 2: Keyword Search Analysis ---
First occurrence found: 'AI'
Starting position: 25
Ending position: 27
Total occurrences of 'AI': 6
```

Section 3: Split the documentation into sentences and words

Refer the previous passage and Write a Python program using the re.split() method to perform the following operations:

- Split the passage into sentences using appropriate punctuation (., ?, !) as delimiters.
- Count and display the total number of sentences.
- Split the passage into words by considering one or more whitespace characters as delimiters.
- Count and display the total number of words.
- Display the list of extracted sentences and words.

ANSWER:

```
import re
```

```
passage = (
    "Artificial Intelligence (AI) is transforming industries across the world. "
    "AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, "
    "and in education to provide personalized learning experiences. Many companies "
    "invest heavily in AI research because AI improves efficiency and enables intelligent "
    "decision-making. As AI continues to evolve, professionals with AI skills are in high demand."
)
```

```
print("--- Section 3: Text Tokenization & Segmentation ---")
```

```
# 1. Split into sentences
sentence_pattern = r'[.?!]\s*'
sentences = re.split(sentence_pattern, passage.strip())
# Remove any empty trailing strings caused by terminal punctuation
sentences = [s for s in sentences if s]
```

```
# 2. Count and display total number of sentences
print(f"Total number of sentences: {len(sentences)}")
```

```
# 3. Split into words using whitespace delimiters
word_pattern = r'\s+'
words = re.split(word_pattern, passage.strip())
```

```
# 4. Count and display total number of words
```

```
print(f"Total number of words: {len(words)}\n")
```

5. Display extracted sentences and words

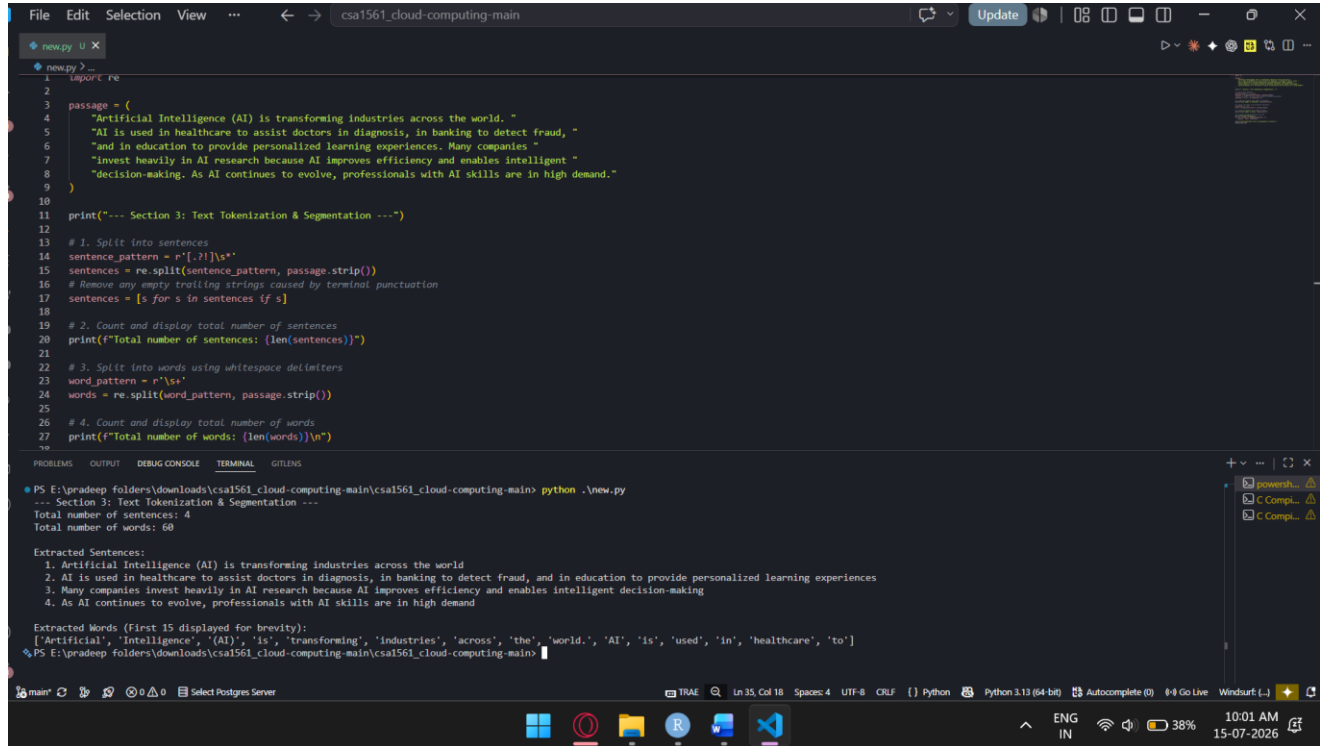
```
print("Extracted Sentences:")
```

```
for idx, sentence in enumerate(sentences, 1):
```

```
    print(f" {idx}. {sentence}")
```

```
print("\nExtracted Words (First 15 displayed for brevity):")
```

```
print(words[:15])
```



```
File Edit Selection View ... csa1561_cloud-computing-main
new.py
1 import re
2
3 passage = {
4     "Artificial Intelligence (AI) is transforming industries across the world. "
5     "AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, "
6     "and in education to provide personalized learning experiences. Many companies "
7     "invest heavily in AI research because AI improves efficiency and enables intelligent "
8     "decision-making. As AI continues to evolve, professionals with AI skills are in high demand."
9 }
10
11 print("--- Section 3: Text Tokenization & Segmentation ---")
12
13 # 1. Split into sentences
14 sentence_pattern = r'[.?!]\s*'
15 sentences = re.split(sentence_pattern, passage.strip())
16 # Remove any empty trailing strings caused by terminal punctuation
17 sentences = [s for s in sentences if s]
18
19 # 2. Count and display total number of sentences
20 print(f"Total number of sentences: {len(sentences)}")
21
22 # 3. Split into words using whitespace delimiters
23 word_pattern = r'\s+'
24 words = re.split(word_pattern, passage.strip())
25
26 # 4. Count and display total number of words
27 print(f"Total number of words: {len(words)}\n")
28
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL GIT LENS
PS E:\pradeep\folders\downloads\csa1561_cloud-computing-main\csa1561_cloud-computing-main> python .\new.py
--- Section 3: Text Tokenization & Segmentation ---
Total number of sentences: 4
Total number of words: 60

Extracted Sentences:
1. Artificial Intelligence (AI) is transforming industries across the world
2. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences
3. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making
4. As AI continues to evolve, professionals with AI skills are in high demand

Extracted Words (First 15 displayed for brevity):
['Artificial', 'Intelligence', 'AI', 'is', 'transforming', 'industries', 'across', 'the', 'world.', 'AI', 'is', 'used', 'in', 'healthcare', 'to']
%PS E:\pradeep\folders\downloads\csa1561_cloud-computing-main\csa1561_cloud-computing-main
```

Section 4: Email and Strong Password Validation

A website requires users to register by providing an Email ID and a Strong Password. Write a Python program using the re module to validate the following inputs.

Validation Criteria:

Email ID

- Must begin with one or more letters or digits.
- May contain ., _, %, +, or - before the @ symbol.
- Must contain a valid domain name.
- Must end with a valid domain extension such as .com, .org, .edu, or .in.

Strong Password

- Must contain at least 8 characters.
- Must contain at least one uppercase letter.
- Must contain at least one lowercase letter.
- Must contain at least one digit.
- Must contain at least one special character from @, #, \$, %, &, !, or *.
- Must not contain any whitespace characters.

ANSWER:

```
import re
```

```
# Define validation functions using regex
```

```
def validate_email(email):
```

```
    pattern = r'^[a-zA-Z0-9][a-zA-Z0-9._%+~]*@[a-zA-Z0-9.-]+\.(com|org|edu|in)$'
```

```
    return bool(re.match(pattern, email))
```

```
def validate_password(password):
```

```
    pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#%$&!*])[\^s]{8,}$'
```

```
    return bool(re.match(pattern, password))
```

```

# Test data
test_emails = [
    "john.doe123@gmail.com", # Valid
    ".johndoe@org.in",      # Invalid: starts with a dot
    "student@university.edu", # Valid
    "admin@domain.net"      # Invalid: .net is not allowed
]

test_passwords = [
    "P@ssw0rd123",          # Valid
    "password123",           # Invalid: missing uppercase & special char
    "PASS#123",              # Invalid: missing lowercase & too short (<8)
    "P@ss word1"             # Invalid: contains whitespace
]

print("--- Section 4: Email & Password Validation ---")
print("Email Validation Results:")
for email in test_emails:
    status = "VALID" if validate_email(email) else "INVALID"
    print(f" {email}: {status}")

print("\nPassword Validation Results:")
for pwd in test_passwords:
    status = "VALID" if validate_password(pwd) else "INVALID"
    print(f" {pwd}: {status}")

```

The screenshot shows a Python IDE with a file named 'new.py'. The code in the file matches the code block above. The terminal output shows the execution results:

```

PS E:\pradeep\downloads\csa1561_cloud-computing-main\csa1561_cloud-computing-main> python .\new.py
--- Section 4: Email & Password Validation ---
Email Validation Results:
'john.doe123@gmail.com': VALID
'.johndoe@org.in': INVALID
'student@university.edu': VALID
'admin@domain.net': INVALID

Password Validation Results:
'P@ssw0rd123': VALID
'password123': INVALID
'PASS#123': INVALID
'P@ss word1': INVALID

```

Section 5: Compile Once, Validate Many

A company receives 1,000 email IDs from its customers and needs to validate each one efficiently. Instead of compiling the same regular expression for every email, write a Python program using the `re.compile()` method to compile the pattern once and reuse it to validate all the email IDs.

Answer:

```

import re
import time

```

```

# 1. Compile the regex pattern ONCE
email_pattern = re.compile(r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$')

```

```

# Simulate a batch of 1,000 email IDs (valid and invalid mixes)
base_emails = [
    "user.name@company.com",
    "invalid-email-at-domain",
    "support+tech@service.org",
    "no_extension@domain.",
    "hr.dept@enterprise.co.in"
]

# Repeat the list 200 times to create exactly 1,000 records
email_batch = base_emails * 200

print("--- Section 5: Pattern Reusability & Performance Optimization ---")
print(f"Total emails to validate: {len(email_batch)}")

valid_count = 0
invalid_count = 0

# Measure execution time for batch validation
start_time = time.time()

# 2. Reuse the compiled pattern across the entire dataset
for email in email_batch:
    if email_pattern.match(email):
        valid_count += 1
    else:
        invalid_count += 1

end_time = time.time()
execution_time = (end_time - start_time) * 1000 # convert to milliseconds

print("Validation Complete!")
print(f" Valid Emails Found : {valid_count}")
print(f" Invalid Emails Found: {invalid_count}")
print(f" Batch Processing Time: {execution_time:.4f} ms")

```

The screenshot shows a Visual Studio Code editor window with a Python script for email validation. The script defines a list of base email patterns, repeats them 200 times to create a batch of 1,000 emails, and then uses a compiled regular expression to validate each email. It counts the number of valid and invalid emails and measures the execution time. The terminal output shows the script running successfully, reporting 600 valid emails, 400 invalid emails, and a batch processing time of 1.9991 ms.

```

File Edit Selection View ... csa1561_cloud-computing-main
new.py X
new.py > ...
1 import re
2 import time
3
4 # 1. Compile the regex pattern ONCE
5 email_pattern = re.compile(r"^[a-zA-Z0-9_+]+\.[a-zA-Z0-9_+]\.[a-zA-Z]{2,}$")
6
7 # Simulate a batch of 1,000 email IDs (valid and invalid mixes)
8 base_emails = [
9     "user.name@company.com",
10    "invalid-email-at-domain",
11    "support+tech@service.org",
12    "no_extension@domain.",
13    "hr.dept@enterprise.co.in"
14 ]
15
16 # Repeat the list 200 times to create exactly 1,000 records
17 email_batch = base_emails * 200
18
19 print("--- Section 5: Pattern Reusability & Performance Optimization ---")
20 print(f"Total emails to validate: {len(email_batch)}")
21
22 valid_count = 0
23 invalid_count = 0
24
25 # Measure execution time for batch validation
26 start_time = time.time()
27
28 # 2. Reuse the compiled pattern across the entire dataset
29 for email in email_batch:
30     if email_pattern.match(email):
31         valid_count += 1
32     else:
33         invalid_count += 1
34
35 end_time = time.time()
36 execution_time = (end_time - start_time) * 1000 # convert to milliseconds
37
38 print("Validation Complete!")
39 print(f" Valid Emails Found : {valid_count}")
40 print(f" Invalid Emails Found: {invalid_count}")
41 print(f" Batch Processing Time: {execution_time:.4f} ms")

```

```

PS E:\pradeep\folders\downloads\csa1561_cloud-computing-main\csa1561_cloud-computing-main> python .\new.py
--- Section 5: Pattern Reusability & Performance Optimization ---
Total emails to validate: 1000
Validation Complete!
Valid Emails Found : 600
Invalid Emails Found: 400
Batch Processing Time: 1.9991 ms
PS E:\pradeep\folders\downloads\csa1561_cloud-computing-main\csa1561_cloud-computing-main>

```

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q. No. / Criteria	Conceptual Understanding	Accuracy of Answer	Application of Knowledge	Completeness of Response	Clarity & Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____